## 1)Design of spectrum Analyzer for Audio Systems

## a) Spectrum analyzer for ECG in MATLAB using FFT and Cepstrum

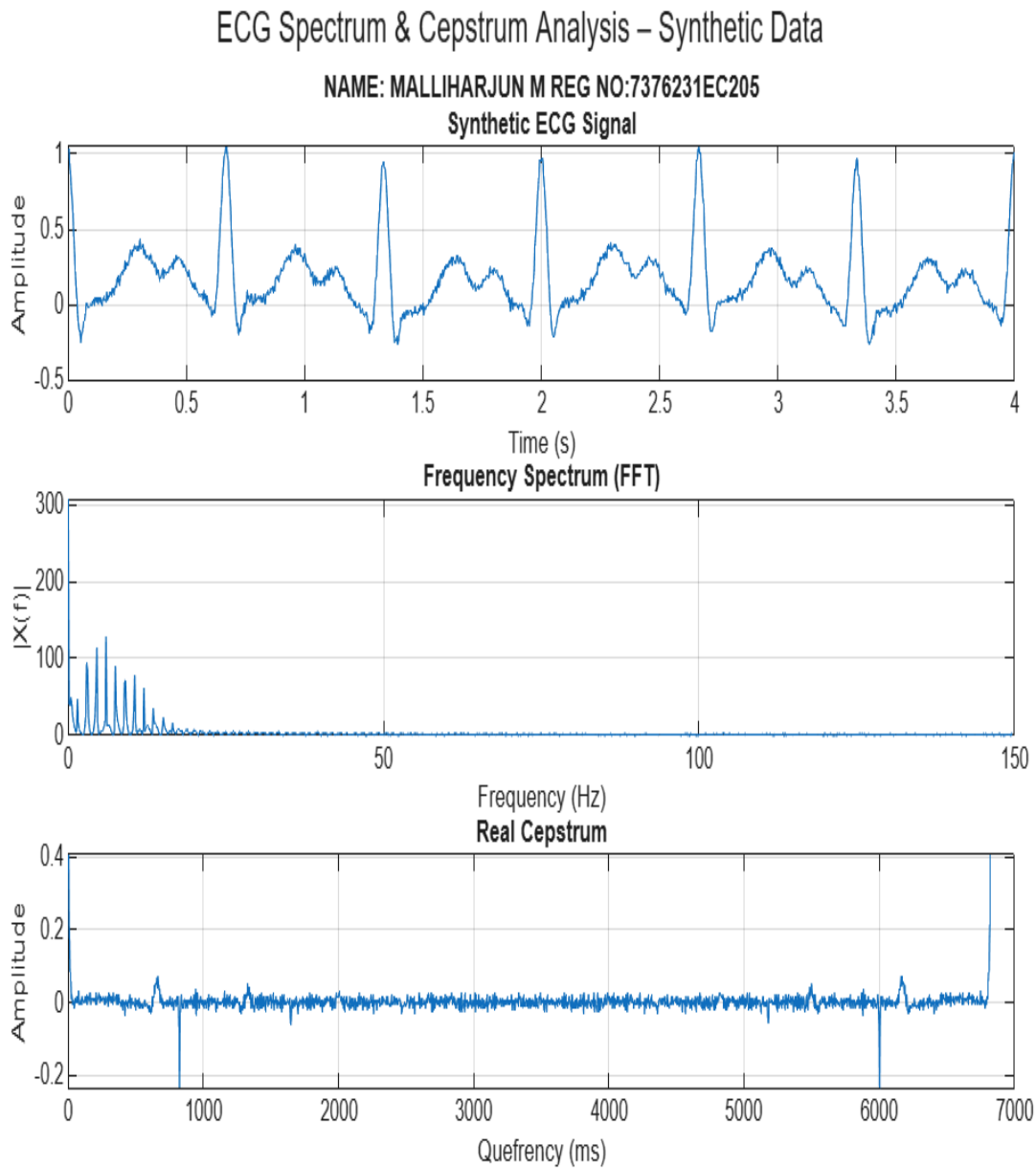
## b) Signal analyzer for the EMG signals in MATLAB using Hilbert Transform and FFT

## 1a)

## program:

```
clc; clear; close all;
fs = 300; dur = 6; t = 0:1/fs:dur;
hr = 90; T0 = 1/(hr/60);
waves = [-0.20 0.25 0.04; -0.05 -0.15 0.015; 0 1 0.020; 0.05 -0.25 0.015; 0.30
0.35 0.060];
ecg = zeros(size(t));
for k = 0:floor(dur/T0)
   R0 = k*T0;
   for w = 1:size(waves,1)
       t0 = R0 + waves(w,1);
       ecg = ecg + waves(w,2)*exp(-0.5*((t-t0)/waves(w,3)).^2);
   end
end
ecg = ecg + 0.05*sin(2*pi*0.5*t) + 0.02*randn(size(t));
N = 2^nextpow2(numel(ecg));
F = fft(ecg, N);
f = fs*(0:N/2-1)/N;
mag = abs(F(1:N/2));
c = real(ifft(log(abs(F)+eps)));
q = (0:N-1)/fs;
figure('Color','w','Position',[200 120 900 550]);
subplot(3,1,1); plot(t, ecg); xlim([0 4]); grid on;
title({'NAME: MALLIHARJUN M REG NO:7376231EC205','Synthetic ECG Signal'});
xlabel('Time (s)'); ylabel('Amplitude');
subplot(3,1,2); plot(f, mag); grid on;
title('Frequency Spectrum (FFT)'); xlabel('Frequency (Hz)'); ylabel('|X(f)|');
subplot(3,1,3); plot(q*1e3, c); grid on;
title('Real Cepstrum'); xlabel('Quefrency (ms)'); ylabel('Amplitude');
sgtitle('ECG Spectrum & Cepstrum Analysis - Synthetic Data');
```

**Output:**

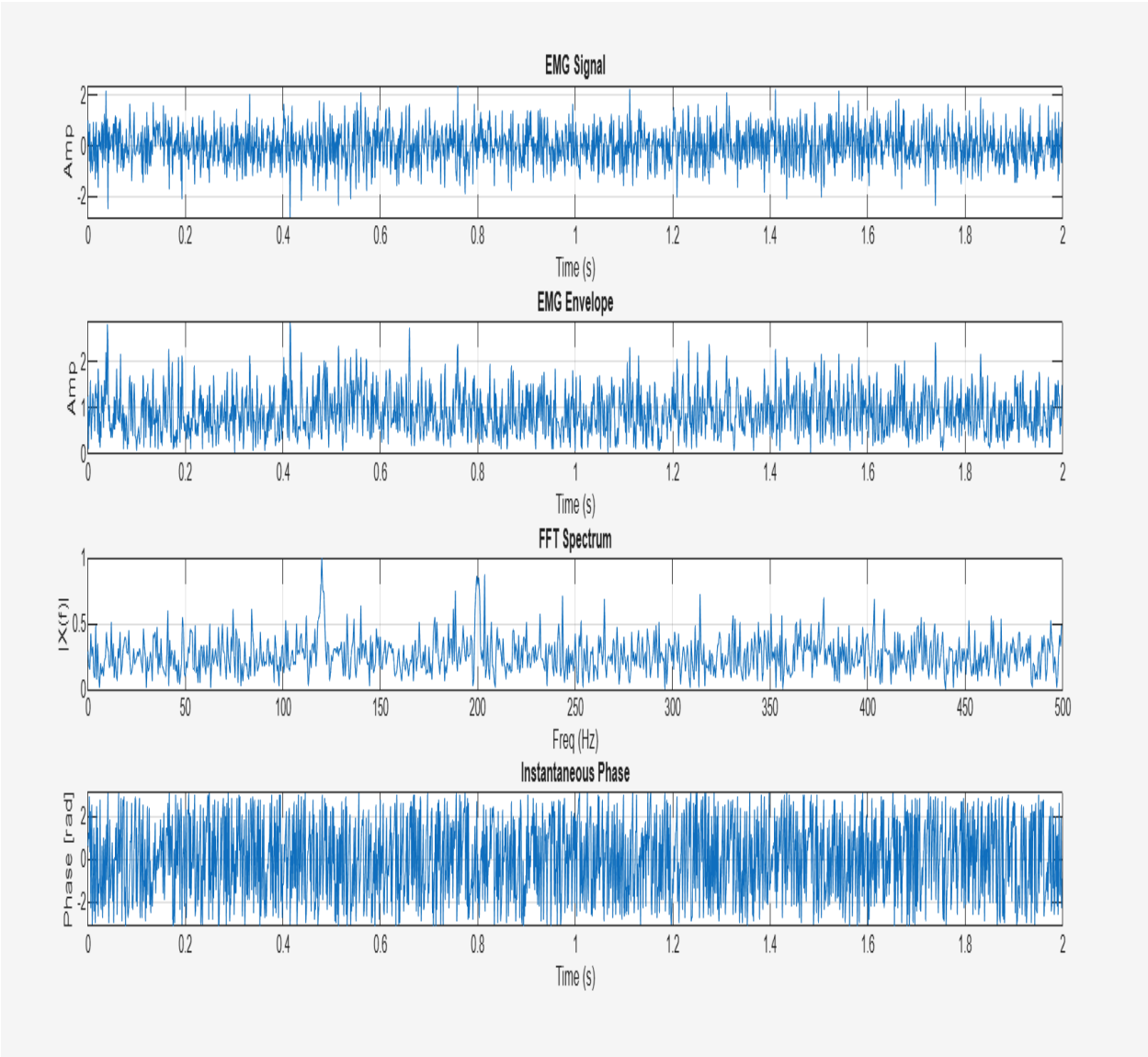

ECG Spectrum & Cepstrum Analysis – Synthetic Data
NAME: MALLIHARJUN M REG NO:7376231EC205
Synthetic ECG Signal
Amplitude
Time (s)
Frequency Spectrum (FFT)
|X(f)|
Frequency (Hz)
Real Cepstrum
Amplitude
Quefrency (ms)

## 1b)

## program:

```
clc; clear; close all;
Fs = 1000; t = 0:1/Fs:2-1/Fs;
emg = 0.7*randn(size(t)) + 0.5*sin(2*pi*120*t).*(t>0.4 & t<0.8) ...
                    + 0.4*sin(2*pi*200*t).*(t>1.2 & t<1.6) ...
                    + 0.3*sin(2*pi*300*t).*(t>0.9 & t<1.1);
analytic_signal = hilbert(emg);
envelope = abs(analytic_signal);
phase = angle(analytic_signal);
f = (0:length(t)-1)*Fs/length(t);
mag = abs(fft(emg))/max(abs(fft(emg)));
figure;
subplot(4,1,1); plot(t, emg);
title('EMG Signal');
xlabel('Time (s)');
ylabel('Amp');
grid on;
subplot(4,1,2);
plot(t, envelope);
title('EMG Envelope');
xlabel('Time (s)');
ylabel('Amp');
grid on;
subplot(4,1,3);
plot(f(1:end/2), mag(1:end/2));
title('FFT Spectrum');
xlabel('Freq (Hz)');
ylabel('|X(f)|');
grid on;
subplot(4,1,4);
plot(t, phase);
title('Instantaneous Phase');
xlabel('Time (s)');
ylabel('Phase [rad]');
grid on;
```

**Output:**


EMG Signal
Amp
Time (s)
EMG Envelope
Amp
Time (s)
FFT Spectrum
|X(f)|
Freq (Hz)
Instantaneous Phase
Phase [rad]
Time (s)

## 2 ) Dual role of noise canceller as enhancer in headphones

## a) Noise canceller for variable SNR of signals with AWGN __ for variable SNRdB 1dB,10dB,15 dB

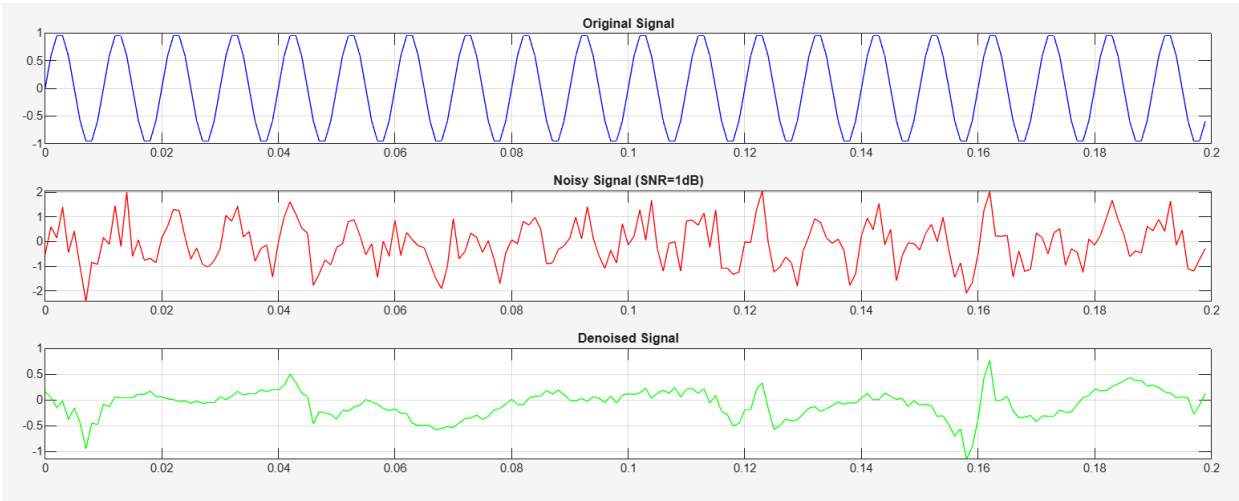
## b) Noise canceller and signal enhancer for ECG signals with AWGN and various SNR

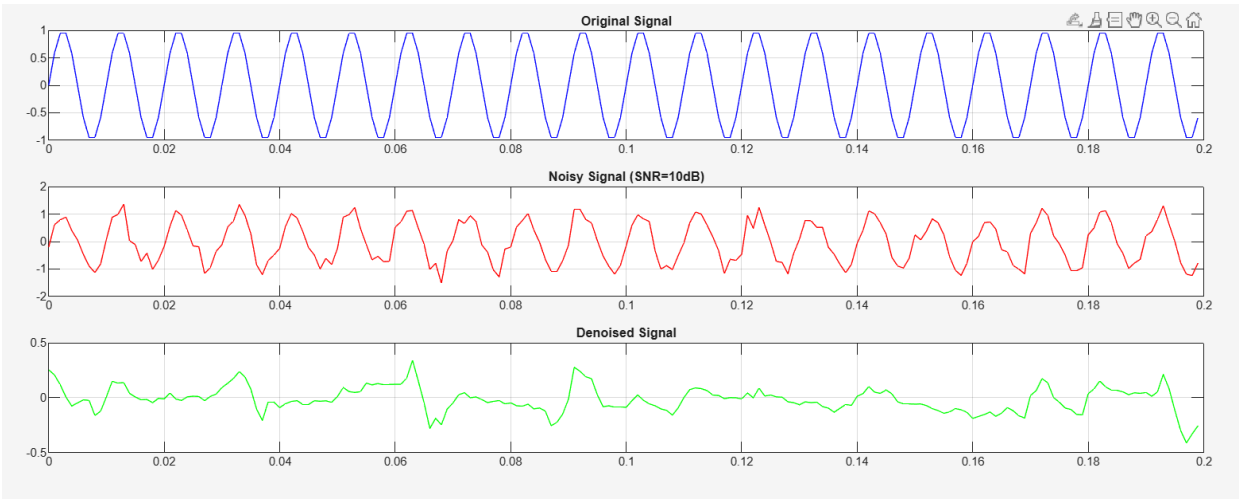
## 2a)

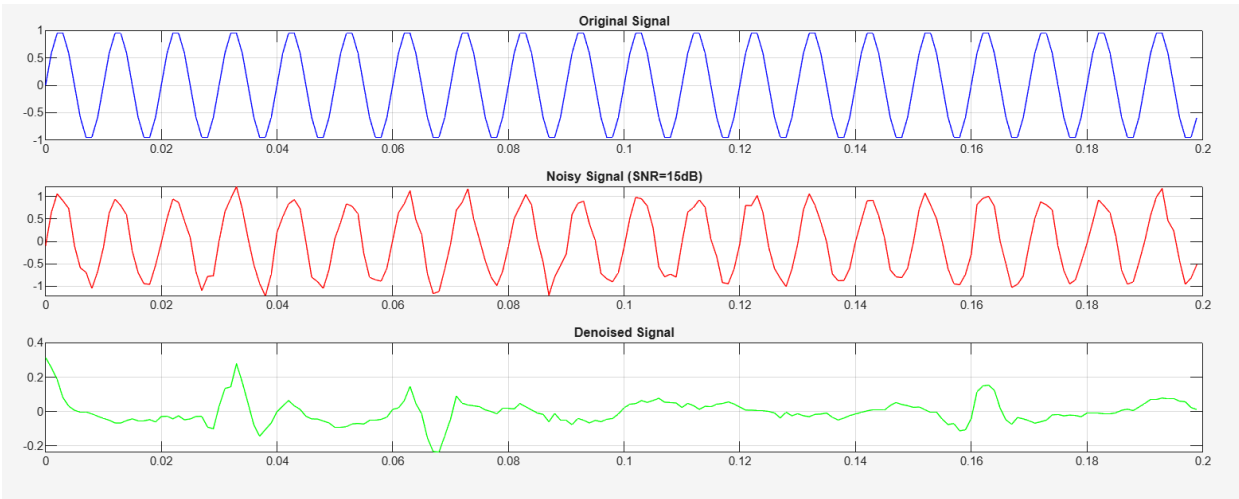
## program:

```
clc; clear; close all;
fs = 1000; t = 0:1/fs:1; x = sin(2*pi*100*t);
SNR_dB = [1, 10, 15]; range = 1:200;
for snr = SNR_dB
   noise = sqrt(mean(x.^2)/10^(snr/10)) * randn(size(x));
   noisy_signal = x + noise;
   denoised_signal = wiener2(noisy_signal, [1 10]);

   figure;
   subplot(3,1,1);
   plot(t(range), x(range),'b');
   title('Original Signal');
   grid on;
   subplot(3,1,2);
   plot(t(range), noisy_signal(range),'r');
   title(['Noisy Signal (SNR=' num2str(snr) 'dB)']);
   grid on;
   subplot(3,1,3);
   plot(t(range), denoised_signal(range),'g');
   title('Denoised Signal');
   grid on;
End
```

**output:**


Original Signal
Noisy Signal (SNR=1dB)
Denoised Signal

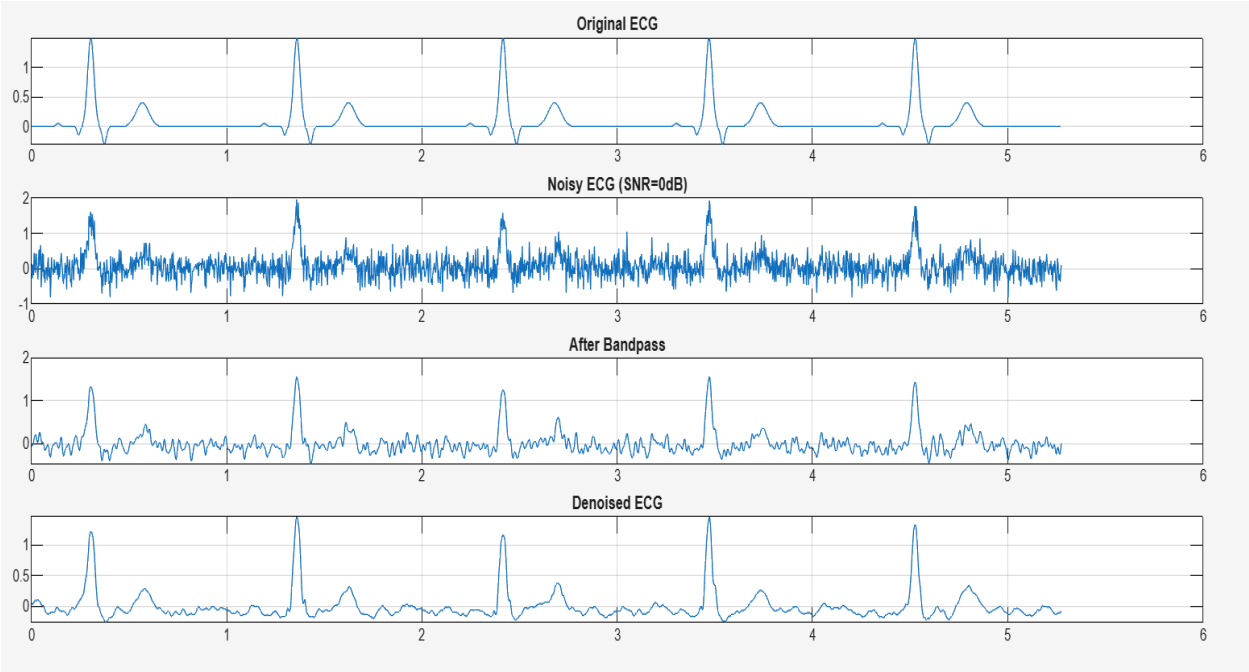

Original Signal
Noisy Signal (SNR=10dB)
Denoised Signal

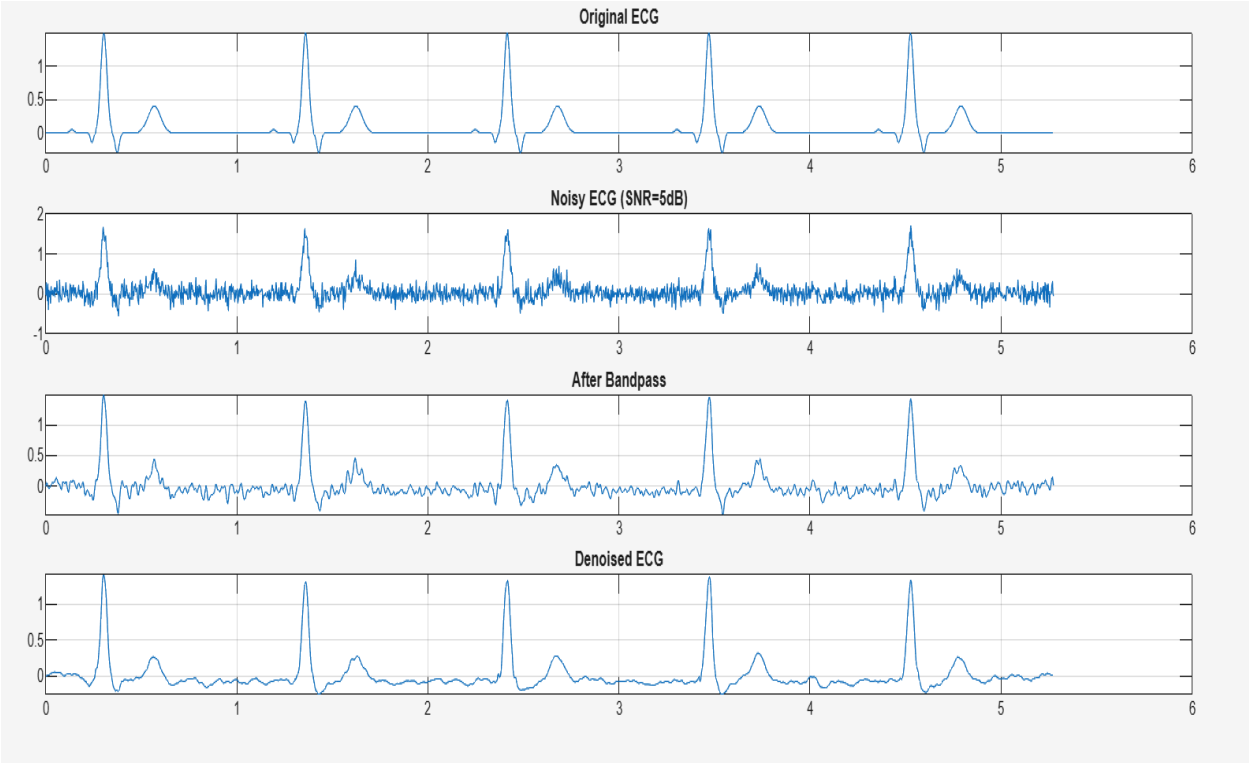

Original Signal
Noisy Signal (SNR=15dB)
Denoised Signal

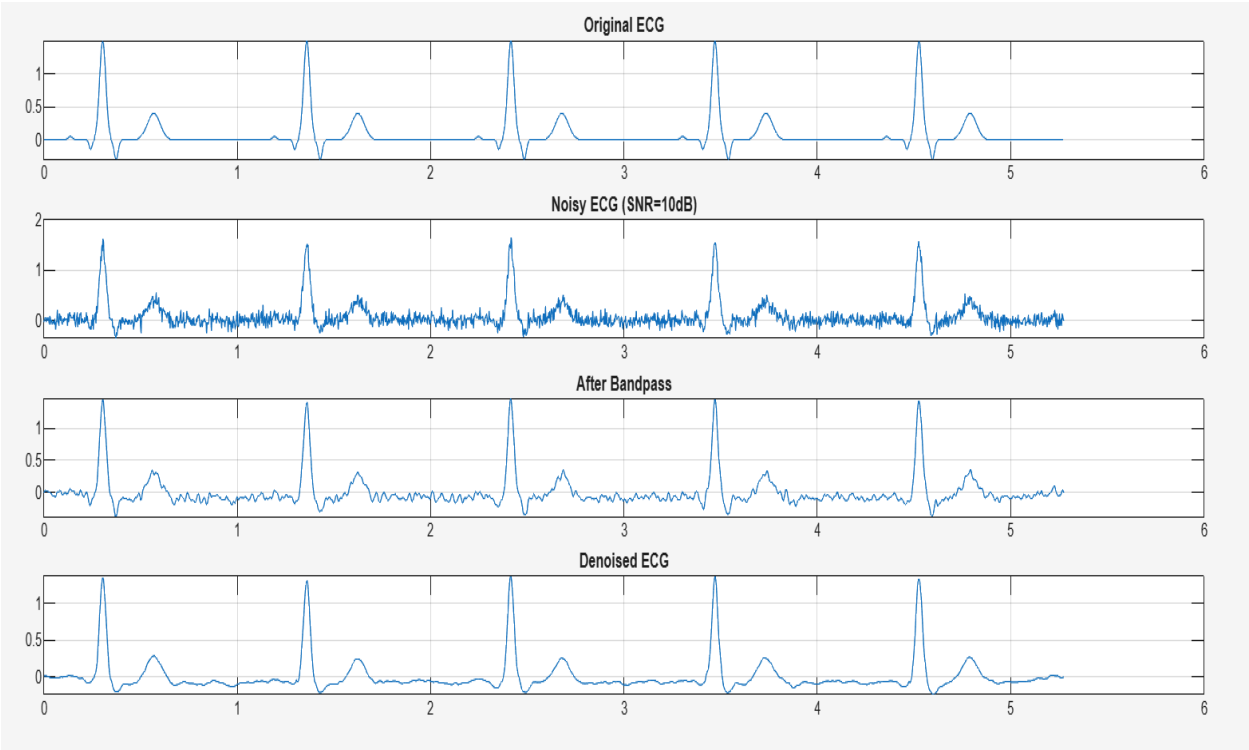
## 2b)

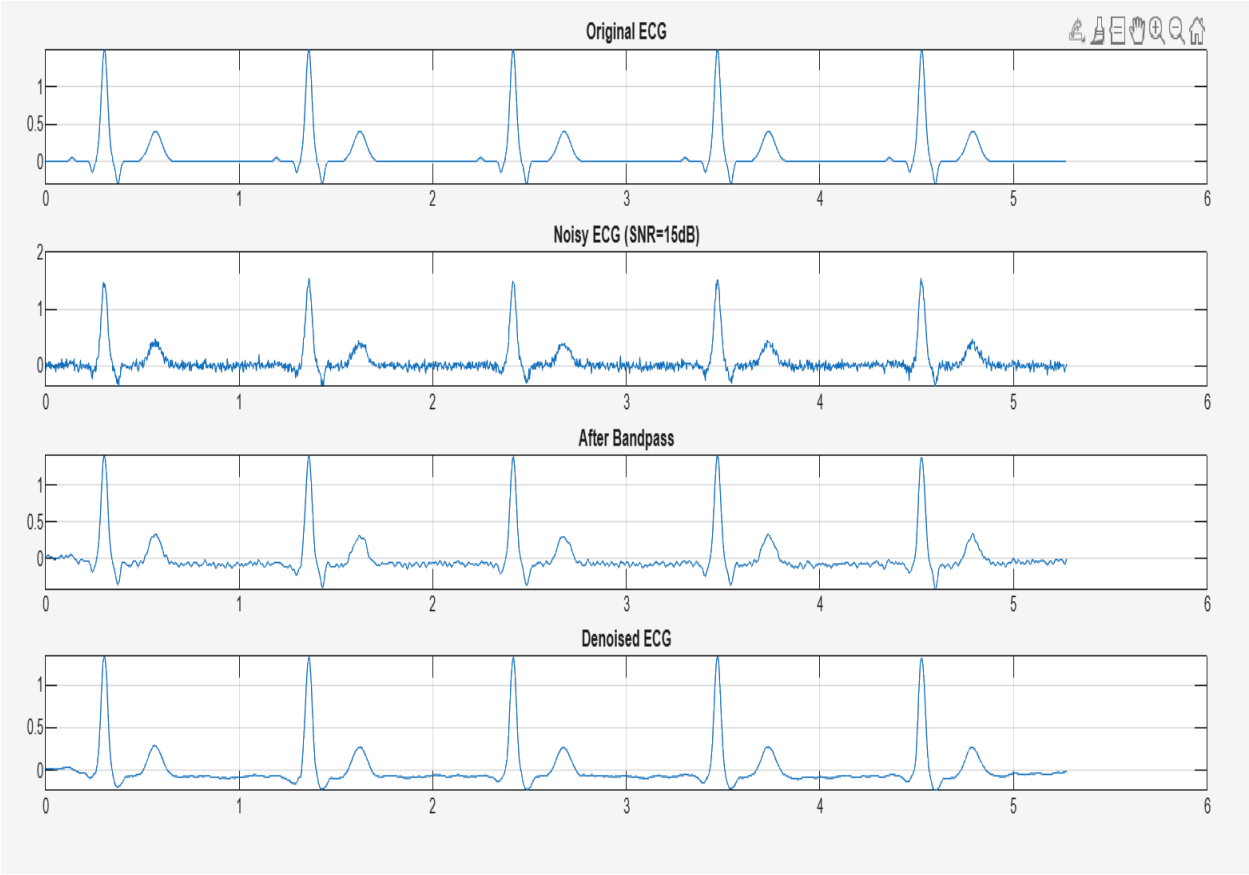
```
clc; clear; close all;
fs = 360;
beat = [zeros(1,40), 0.05*gausswin(20)', zeros(1,20), ...
       -0.15*gausswin(15)', 1.5*gausswin(30)', -0.3*gausswin(20)', ...
       zeros(1,30), 0.4*gausswin(60)', zeros(1,145)];
ecg_clean = repmat(beat,1,5);
t = 0:1/fs:(length(ecg_clean)-1)/fs;
SNR_dB = [0, 5, 10, 15];
signal_power = mean(ecg_clean.^2);
bp =
designfilt('bandpassiir','FilterOrder',4,'HalfPowerFrequency1',0.5,'HalfPowerF
requency2',45,'SampleRate',fs);
for snr = SNR_dB
   noise = sqrt(signal_power/10^(snr/10)) * randn(size(ecg_clean));
   ecg_noisy = ecg_clean + noise;
   ecg_filt = filtfilt(bp, ecg_noisy);
   ecg_denoised = wiener2(ecg_filt, [1 20]);

   figure;
   subplot(4,1,1);
   plot(t, ecg_clean);
   title('Original ECG');
   grid on;
   subplot(4,1,2);
   plot(t, ecg_noisy);
   title(['Noisy ECG (SNR=' num2str(snr) 'dB)']);
   grid on;
   subplot(4,1,3);
   plot(t, ecg_filt);
   title('After Bandpass');
   grid on;
   subplot(4,1,4);
   plot(t, ecg_denoised);
   title('Denoised ECG');
   grid on;
End
```

**output:**


Original ECG
Noisy ECG (SNR=0dB)
After Bandpass
Denoised ECG


Original ECG
Noisy ECG (SNR=5dB)
After Bandpass
Denoised ECG


Original ECG
Noisy ECG (SNR=10dB)
After Bandpass
Denoised ECG


Original ECG
Noisy ECG (SNR=15dB)
After Bandpass
Denoised ECG

## 3) Peak signal analyser with IIR filters for detecting pulse signal in Pulse Oximeter

## a) Peak signal (ECG, EMG, EEG) analyser with IIR filters. (5th Order Butterworth, 2nd or 3rd order Chebyshev)

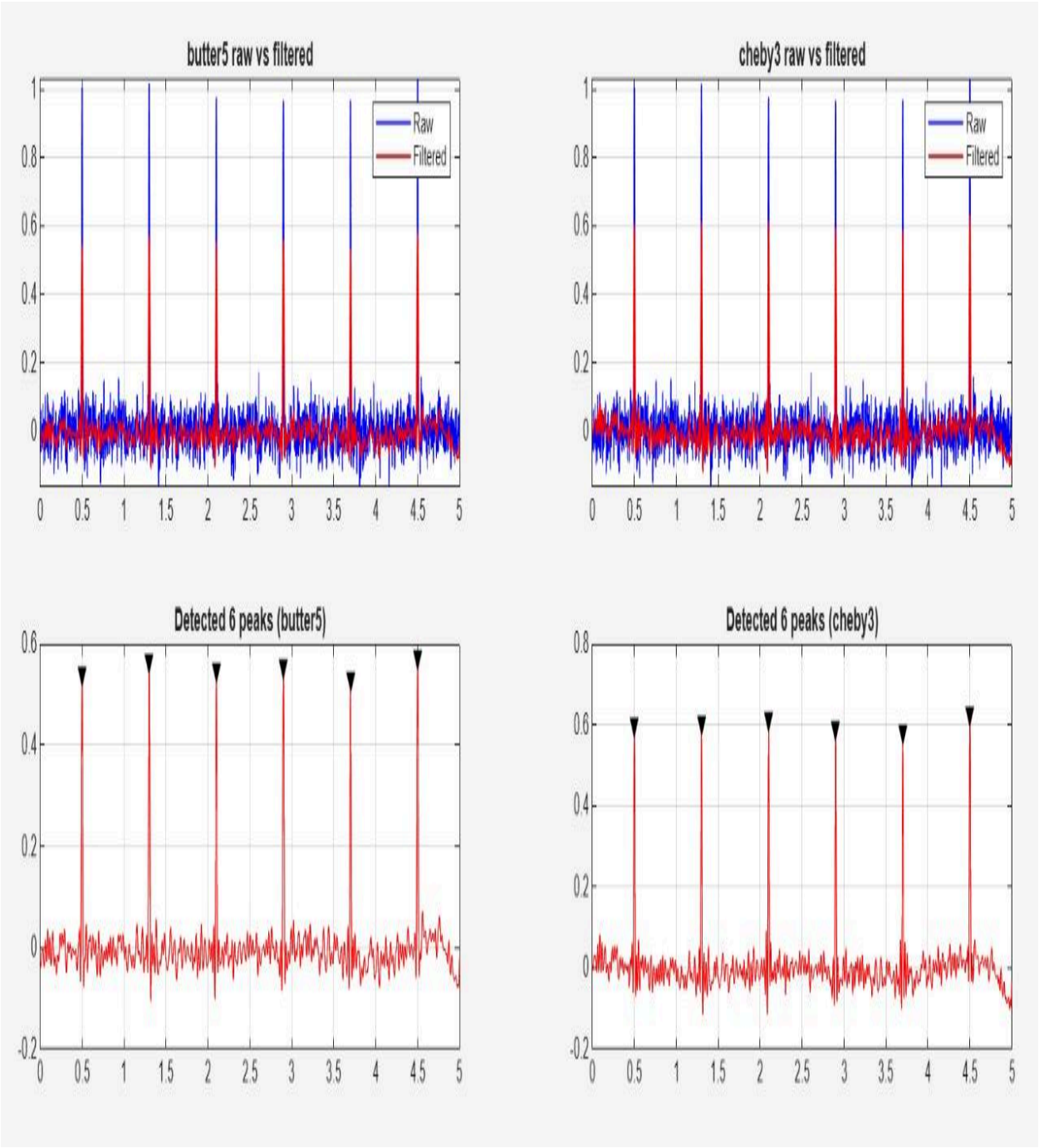
## b) Time domain peak signal detection and analysis using 2nd order FIR filter

## 3a)

```
clc; clear; close all;
fs = 360; t = (0:1/fs:5)';
sig = 0.05*randn(size(t));
r_locs = 0.5:0.8:5;
for rt = r_locs
   idx = round(rt*fs);
   if idx+3 <= length(sig)
       sig(idx:idx+3) = sig(idx:idx+3) + [0.8 1 0.6 0.3]';
   end
end
[pk1, tm1] = runAnalyser(sig, fs, 'butter5');
[pk2, tm2] = runAnalyser(sig, fs, 'cheby3');
fprintf('Butterworth detected %d peaks\n', numel(pk1));
fprintf('Chebyshev detected %d peaks\n', numel(pk2));
function [peaks, peakTimes] = runAnalyser(x, fs, filterType)
Wn = [0.5 40]/(fs/2);
switch lower(filterType)
   case 'butter5', [b,a] = butter(5, Wn, 'bandpass');
   case 'cheby3', [b,a] = cheby1(3, 0.5, Wn, 'bandpass');
end
filtered = filtfilt(b,a,double(x));
```

```
[peaks, locs] =
findpeaks(filtered,'MinPeakProminence',0.3*(max(filtered)-min(filtered)),...
   'MinPeakDistance',round(0.3*fs));
peakTimes = (locs-1)/fs;
t = (0:length(x)-1)/fs;
persistent firstRun;
if isempty(firstRun), figure('Name','ECG Filter Comparison'); firstRun = true;
end
col = strcmpi(filterType,'cheby3')+1;
subplot(2,2,col); plot(t,x,'b',t,filtered,'r','LineWidth',1.1);
legend('Raw','Filtered'); grid on; title([filterType ' raw vs filtered']);
subplot(2,2,col+2);
plot(t,filtered,'r',peakTimes,peaks,'kv','MarkerFaceColor','k');
title(sprintf('Detected %d peaks (%s)',numel(peaks),filterType)); grid on;
End
```

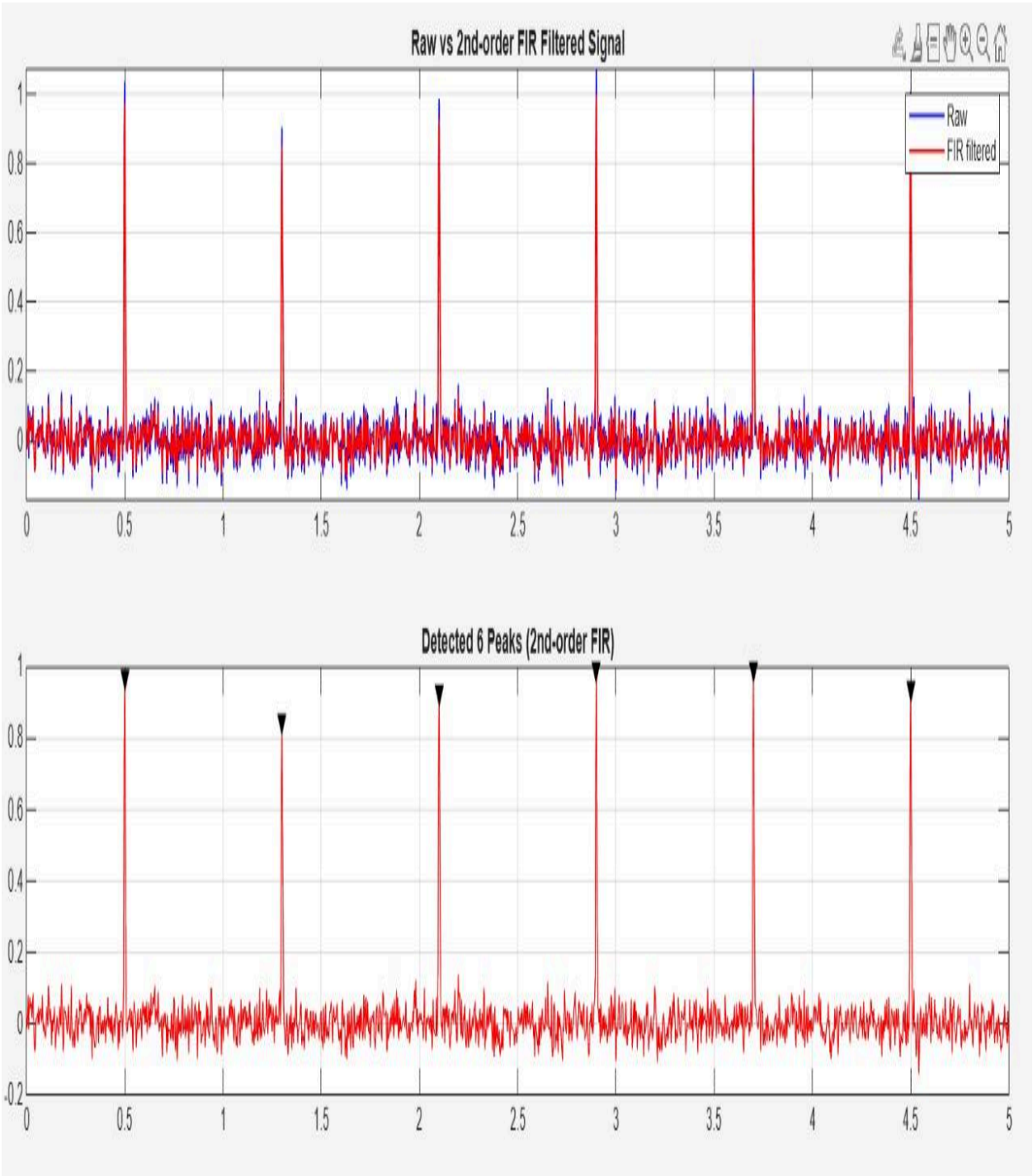
**Output:**


butter5 raw vs filtered
Raw
Filtered
cheby3 raw vs filtered
Raw
Filtered
Detected 6 peaks (butter5)
Detected 6 peaks (cheby3)

## 3b)

```
clc; clear; close all;
fs = 360; t = (0:1/fs:5)';
sig = 0.05*randn(size(t));
r_locs = 0.5:0.8:5;
for rt = r_locs
   idx = round(rt*fs);
   if idx+3 <= length(sig)
       sig(idx:idx+3) = sig(idx:idx+3) + [0.8 1 0.6 0.3]';
   end
end
d =
designfilt('bandpassfir','FilterOrder',2,'CutoffFrequency1',0.5,'CutoffFrequen
cy2',40,'SampleRate',fs);
filt_sig = filtfilt(d.Coefficients,1,sig);
[peaks, locs] =
findpeaks(filt_sig,'MinPeakProminence',0.3*range(filt_sig),'MinPeakDistance',r
ound(0.3*fs));
peakTimes = (locs-1)/fs;
subplot(2,1,1); plot(t,sig,'b',t,filt_sig,'r','LineWidth',1.1);
legend('Raw','FIR filtered'); grid on;
title('Raw vs 2nd-order FIR Filtered Signal');
subplot(2,1,2);
plot(t,filt_sig,'r',peakTimes,peaks,'kv','MarkerFaceColor','k'); grid on;
title(sprintf('Detected %d Peaks (2nd-order FIR)',numel(peaks)));
fprintf('2nd-order FIR filter detected %d peaks\n',numel(peaks));
```

**output:**


Raw vs 2nd-order FIR Filtered Signal
Raw
FIR filtered
1
0.8
0.6
0.4
0.2
0
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
5
Detected 6 Peaks (2nd-order FIR)
1
0.8
0.6
0.4
0.2
0
-0.2
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
5

## 4)Ultrasonic Image Edge detection in frequency domain for analyzing tumor size

## a) Image edge detection in frequency domain a) 5th or above Butterworth High pass filter

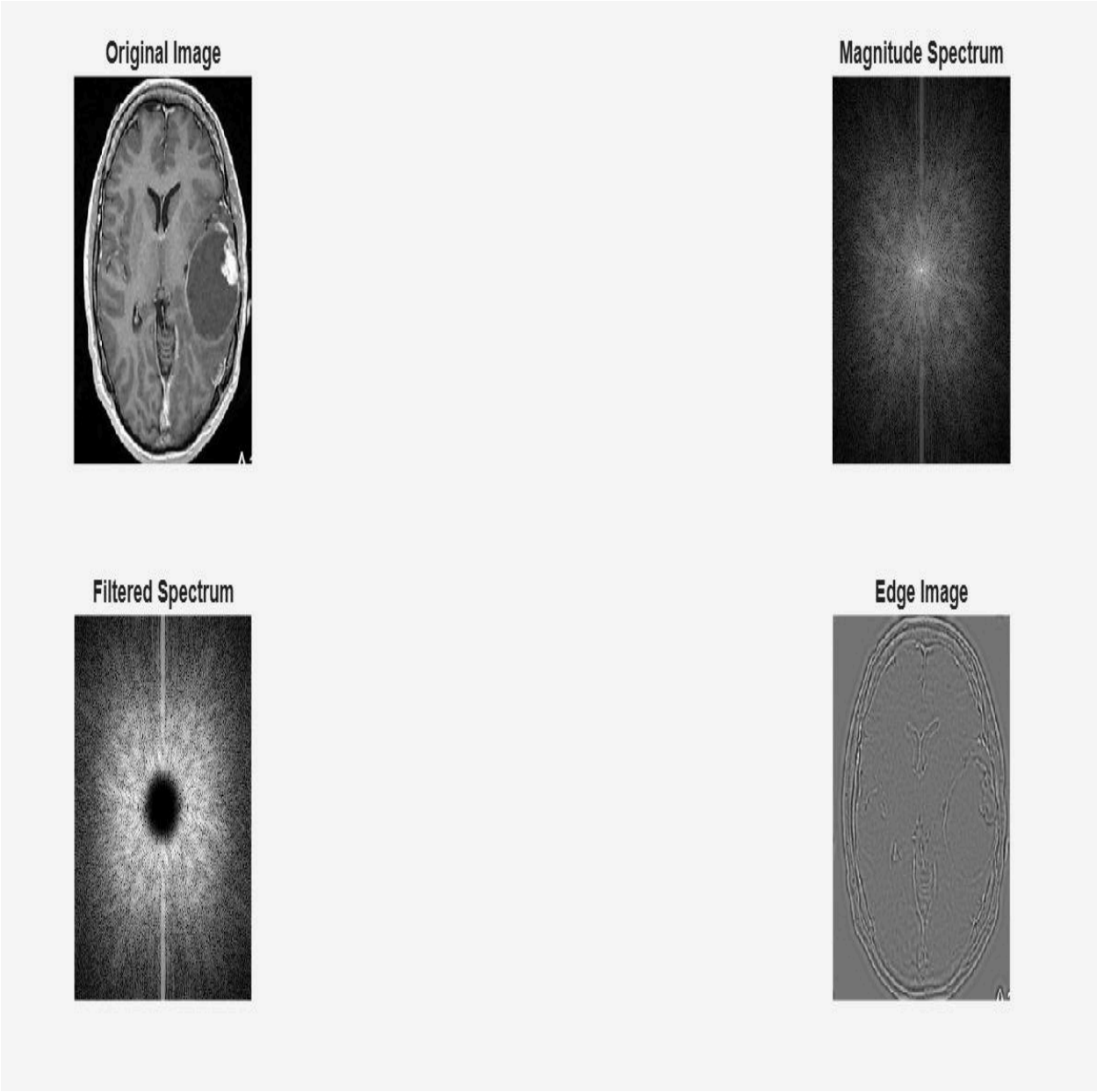
## b) Reconstruction of edges for SNR, PSNR

## c) Inverse filtering using Butterworth

## 4a)

```
clc; clear; close all;
I = imread('ultrasound_tumor.png');
if size(I,3)==3, I = rgb2gray(I); end
I = im2double(I);
F = fftshift(fft2(I));
[M,N] = size(I); [U,V] = meshgrid(1:N,1:M);
D0 = 30; n = 5;
D = sqrt((U-N/2).^2 + (V-M/2).^2);
H = 1 ./ (1 + (D0./D).^(2*n));
G = F .* H;
I_edge = real(ifft2(ifftshift(G)));
figure;
subplot(2,2,1); imshow(I,[]); title('Original Image');
subplot(2,2,2); imshow(log(1+abs(F)),[]); title('Magnitude Spectrum');
subplot(2,2,3); imshow(log(1+abs(G)),[]); title('Filtered Spectrum');
subplot(2,2,4); imshow(I_edge,[]); title('Edge Image');
```

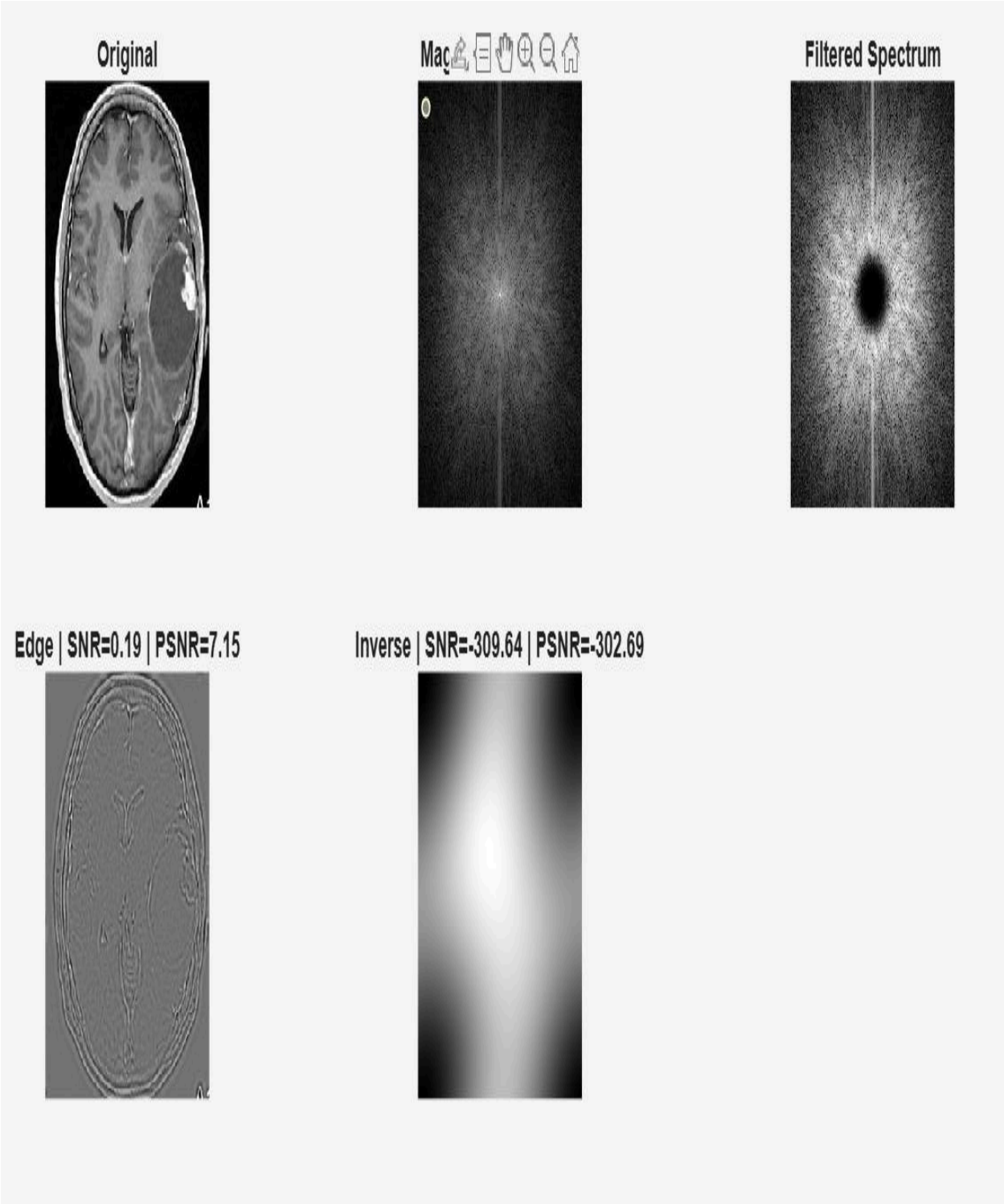
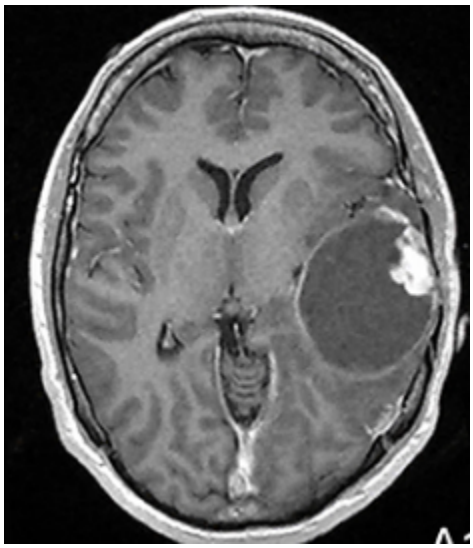
**output:**


Original Image
Magnitude Spectrum
Filtered Spectrum
Edge Image

# 4b&c)

```
clc; clear; close all;
I = im2double(rgb2gray(imread('ultrasound_tumor.png')));
[M, N] = size(I);
F = fftshift(fft2(I));
[U, V] = meshgrid(1:N, 1:M);
D = sqrt((U-N/2).^2 + (V-M/2).^2);
D(D==0) = eps;
D0 = 30; n = 5;
H = 1 ./ (1 + (D0./D).^(2*n));
I_edge = real(ifft2(ifftshift(F .* H)));
I_inv  = real(ifft2(ifftshift(F ./ (H + eps))));
SNR_edge = snr(I(:), I(:)-I_edge(:));
PSNR_edge = psnr(I_edge, I);
SNR_inv  = snr(I(:), I(:)-I_inv(:));
PSNR_inv = psnr(I_inv, I);
figure('Position',[100 100 1200 500]);
subplot(2,3,1); imshow(I,[]); title('Original');
subplot(2,3,2); imshow(log(1+abs(F)),[]); title('Magnitude Spectrum');
subplot(2,3,3); imshow(log(1+abs(F.*H)),[]); title('Filtered Spectrum');
subplot(2,3,4); imshow(I_edge,[]); title(['Edge | SNR='
num2str(SNR_edge,'%.2f') ' | PSNR=' num2str(PSNR_edge,'%.2f')]);
subplot(2,3,5); imshow(I_inv,[]); title(['Inverse | SNR='
num2str(SNR_inv,'%.2f') ' | PSNR=' num2str(PSNR_inv,'%.2f')]);
```

**output:**


Original
Mag
Filtered Spectrum
Edge | SNR=0.19 | PSNR=7.15
Inverse | SNR=-309.64 | PSNR=-302.69

## Imported image for 4th experiment: